Project 02 — Paths, query parameters and request schemas

Theory, answers, design decision, scope and references

APIs: 24 Build-It-Yourself Projects

5 November 2026 • FastAPI 0.162.0, Pydantic 2.14.2 (core 2.50.2), Starlette 1.10.0, uvicorn 0.54.0, Python 3.13.18

How to read this. Every claim about *our* API was checked against the running code (TestClient, or a live uvicorn server called with curl on 2026-11-05; raw output in `evidence/live-2026-11-05.txt`). Claims about libraries and standards link to the source in Section 11. Where something was measured rather than read, the text says so.

Contents

1 The boundary: types are a promise to clients **2**

2 Missing, null, invalid and zero are four different things **2**

3 Money is not a float **2**

4 Coercion: lax vs strict **3**

5 Extra fields and mass assignment **4**

6 Path parameters: 422 vs 404 **4**

7 Internal vs public models **5**

8 The workbook's four questions **5**

9 Design decision **6**

10 Scope of the project **6**

11 References **7**

1 The boundary: types are a promise to clients

An API's input types are not internal details. As soon as one client sends `"quantity": "3"` and gets a 200, that spelling is part of the contract, whatever the documentation says. This project therefore validates *everything* at the boundary, before the handler runs, and writes down every case where it still coerces.

FastAPI decides where each value comes from by its declaration:

Declaration	Source	In our API
name inside `{...}` in the path	path segment	`product_id: UUID`
plain type (`str`, `int`) not in the path	query string	none in P02 (P01's `name`)
Pydantic model	JSON body	`request: QuoteRequest`

The first element of an error's location `loc` follows the same split: `path`, `query` or `body`, followed by the field name.

2 Missing, null, invalid and zero are four different things

The workbook's central claim. Measured on the live server for `quantity`:

Body	Status	`type`	Meaning
`{"unit_price":"0.10"}`	422	`missing`	the client did not say anything
`"quantity": null`	422	`int_type`	the client said "no value"
`"quantity": 2.5`	422	`int_type`	a value of the wrong type
`"quantity": 0`	422	`greater_than_equal`	right type, outside the range
`"quantity": 3`	200	—	valid

Required vs optional (R01). In Pydantic a field is required exactly when it has no default. `quantity: int = Field(ge=1)` is still required; `Field(1, ge=1)` would silently make 1 the default, and an absent quantity would be accepted.

Null vs absent (R02). An optional field written as `x: int | None = None` would merge the two cases again, because both end up as `None`. When the difference matters (for example in a PATCH, "leave unchanged" vs "clear"), Pydantic records which fields were actually sent in `model.model_fields_set`. P02 has no optional fields, so both cases are rejected, with different error types.

3 Money is not a float

A binary float cannot store most decimal fractions exactly. Measured in Python 3.13:

```
>>> 0.1 * 3
0.30000000000000004
>>> Decimal(0.1)        # the float's real value
0.1000000000000000055511151231257827021181583404541015625
>>> Decimal("0.10") * 3
Decimal('0.30')
```

JSON itself has only one number type, and RFC 8259 §6 warns that implementations commonly

read numbers as IEEE 754 doubles. So the price travels as a *string* (`"0.10"`), and the server builds a `Decimal` from the exact digits.

3.1 Rounding is a policy, not a default

`quantize(Decimal("0.01"), rounding=...)` rounds to exactly two places. Python offers eight rounding modes, and its default is *half even*:

Value	`ROUND_HALF_UP`	`ROUND_HALF_EVEN`	Note
0.125	0.13	0.12	2 is even, so half even goes down
0.135	0.14	0.14	4 is even, so half even goes up
`round(2.675, 2)`	2.67 (a float, really 2.67499...)		why we never round floats

We chose half up because it matches what people expect on a receipt, and pinned it with a test (`0.125` → `"0.13"`). Half even reduces bias over many roundings and is common in accounting. Either is defensible. Leaving the choice implicit is not.

3.2 Bounds protect the arithmetic

A `Decimal` operation runs in a *context* with a precision of 28 significant digits by default:

```
>>> Decimal("1E+100").quantize(Decimal("0.01"))   # needs 103 digits
decimal.InvalidOperation: [<class 'decimal.InvalidOperation'>]
```

Before R04 our API turned that into a **500** (found by probing, see EVIDENCE.md). With `max_digits=12, decimal_places=4` the largest total is $99\,999\,999.9999 \times 1000 \approx 10^{11}$, about 15 digits at most, so `quantize` can no longer fail.

> A number with no upper bound has not been validated. Bounds are needed for the arithmetic, not only for the business rule.

4 Coercion: lax vs strict

Pydantic validates in *lax* mode by default: it converts values when the conversion seems safe. For an `int` field, `"3"` and `3.0` become `3`, and `2.5` is rejected (`int_from_float`). In *strict* mode, only a real integer is accepted. Our choices, all observed live:

Input	Result	Reason
`quantity: "3", 3.0`	422	`strict=True`: a count is an integer, nothing else
`unit_price: "0.10"`	200	the intended form; must stay lax to parse a string
`unit_price: 0.1` (number)	200	lax; the client has already lost precision
`"1E1"`, `" 0.10 "`, `"+0.10"`, `"1_0"`	200	Python's `Decimal()` grammar accepts all of them
Arabic-Indic digits (U+0661 U+0662)	200, total 12.00	`Decimal()` accepts any Unicode decimal digit
`"-0"`	200, `"-0.00"`	**open bug**: $-0 \geq 0$ is true, and the sign survives
`true`, `"0x10"`	422	not a decimal

The trade-off. RFC 9413 (*Maintaining Robust Protocols*) argues against the old "be liberal in what you accept" rule. Every tolerated variant becomes something clients depend on, and you can never remove it without breaking them. Strictness has the opposite cost: clients that send `"3"` break *today*. We chose strict where coercion could change meaning (a count) and documented the lax cases we kept.

One side effect we measured: `strict=True` changed the error for `2.5` from `int_from_float` to `int_type`. Tests that assert on the library's wording break on such changes. Ours assert on status and `loc`.

5 Extra fields and mass assignment

Pydantic's `extra` setting has three values: `"ignore"` (the default: unknown keys are dropped silently), `"allow"` (they are kept on the model) and `"forbid"` (422 `extra_forbidden`).

- **Ignore hides client bugs.** Our M1 typo `units_price` produced only "`unit_price` missing". With `forbid`, the client gets both "`unit_price` missing" and "`units_price` not permitted", and the typo is obvious.
- **Ignore misleads clients.** A client sending `"discount": "0.5"` receives a full-price quote with a 200, and may believe the discount was applied.
- **Allow invites mass assignment.** If extra keys are kept and copied into storage, a client can set fields it should never control (`is_admin`, `price`). OWASP lists this under API3:2023, *Broken Object Property Level Authorization*.

Measured limit: `extra="forbid"` lives on the body model, so unknown *query* parameters are still ignored (`?admin=true` → 200). FastAPI can also forbid extra query parameters by declaring them as a Pydantic model with `Query()`. We did not do this in P02.

6 Path parameters: 422 vs 404

`product_id: UUID` makes the path segment go through Python's `UUID()` parser before the handler runs.

- `/products/not-a-uuid` → **422** `uuid_parsing` at `["path","product_id"]`: this can *never* identify a product. It's a client error.

- `/products/00000000-0000-4000-8000-000000000000` → **404**: well-formed, but nothing is stored under it.

RFC 9110 defines 404 as "the origin server did not find a current representation for the target resource" (§15.5.5) and 422 as "unable to process the contained instructions" (§15.5.21). Returning 404 for garbage would hide client bugs. Returning 422 for a missing product would blame the client for a correct request.

A lesson from testing: `test_unknown_but_valid_product_id_is_404` passed *before the route existed*, because every unknown path is a 404. A test is only meaningful next to one that proves the route works (the 200 test).

Accepted spellings (measured): uppercase, no hyphens, `{braces}` and `urn:uuid:` all reach the same product. The response is always the canonical lowercase hyphenated form from RFC 9562. Because the dict keys are `UUID` objects, not strings, the lookup is case-independent. P15 (caching) will need to decide whether these are one URL or many.

7 Internal vs public models

R06 separates two schemas for one resource:

```
class ProductRecord(BaseModel):     # internal: what we store
    id: UUID; name: str; supplier_password: str

class ProductResponse(BaseModel):   # public: what clients may see
    id: UUID; name: str

@app.get("/products/{product_id}", response_model=ProductResponse)
def get_product(product_id: UUID):
    ...
    return product                  # the whole record, on purpose
```

FastAPI re-validates the return value into `response_model`, which drops undeclared fields. The handler returns the risky object deliberately, so the test shows that the *schema* is the safety net, not the handler author's memory.

Intentional failure (measured). Without `response_model`, FastAPI serialized the whole record: 2 tests went red and printed the canary password. Restored, 22 passed. A precondition test proves the secret really is stored; otherwise "the secret is absent" would pass trivially.

What `response_model` does not cover. Headers (P01's leak experiment), logs, and the 422 `input` echo. For a `missing` error, `input` is the *whole request body*, so anything sensitive in it is sent back. This is carried to P08.

8 The workbook's four questions

1. What information crosses the contract boundary? In: a JSON object with exactly `unit_price` (decimal string) and `quantity` (integer), or a UUID in the path. Out: `total` as a 2-place decimal string, or `id` + `name`. Errors: 422 with `type` + `loc` per problem, or 404. Nothing else (`supplier_password`, stack traces) crosses.

2. Which invariant would survive a framework change? "The handler only ever sees values inside the contract", and "the total is `unit_price` × `quantity` rounded half up to cents, as a

decimal string". The tests are written as HTTP requests and responses, so they would still apply to another framework. Only the `type` strings are Pydantic-specific.

3. Which failure could hide behind a successful status? Several did: `"quantity": "3"` → 200 (silent coercion); `"discount"` ignored → 200 at full price; `"-0"` → 200 `"-0.00"`; `?admin=true` ignored → 200; and the 404 test, which was green while the route didn't exist.

4. What had to be measured rather than read? The 500 from `"1e100"`; exactly which spellings `Decimal()` and `UUID()` accept (underscores and Unicode digits are not obvious from the docs); that `strict` changes error types; and that `response_model` filters a returned model of a *different* class.

9 Design decision

Chosen: declarative Pydantic constraints. `Field(ge, le, max_digits, decimal_places, strict)` + `extra="forbid"` + a separate response model. *Why:* validation is visible next to the field, runs before the handler, and gives structured errors (`type`, `loc`) for free. *Cost:* we inherit the library's lax parsing (`Decimal()` grammar), and the `type` strings are framework-specific.

Alternative: a strict string type for `unit_price`, converted to `Decimal` afterwards:

```
unit_price: str = Field(pattern=r"^\d{1,8}(\.\d{1,4})?$")
```

This would reject numbers, exponents, spaces, `+`, `_`, non-ASCII digits and `-0` all at once. *Cost:* a second parsing step, and existing clients sending `0.1` as a number would break.

What would make me switch: the first time a lax spelling causes a real disagreement (for example a downstream system that rejects `"1E1"`), or before a public release, while no client depends on the lax forms yet.

10 Scope of the project

In scope (built and tested):

- Two routes: `POST /quotes` (JSON body) and `GET /products/{product_id}` (path).
- R01 required fields, R02 null vs absent, R03 decimal arithmetic + half-up rounding + strict integers, R04 numeric and digit bounds, R05 unknown body fields rejected, R06 internal fields kept out of responses.
- T01–T06, two boundary pairs, two regressions (`1e100`, the `units_price` typo), one intentional failure: 22 tests.
- A live check of 36 requests against uvicorn, and documented coercion rules.

Deliberately out of scope:

- Persistence: `PRODUCTS` is an in-memory dict (P05).
- Custom error format: we keep FastAPI's 422 shape (P08).
- Query-parameter models and unknown query parameters (noted as open).
- Optional request fields and PATCH semantics (P03).
- Authentication and per-user visibility of fields (P09, P10).

- Latency measurement (methodology in P01), concurrency, CI.

Known open items: `"-0"` → `"-0.00"`; lax `unit_price` spellings; unknown query parameters ignored; 422 echoes `input`; `HEAD` → 405 (from P01).

11 References

Standards

- RFC 9110, *HTTP Semantics*: §15.5.5 404 Not Found, §15.5.21 422 Unprocessable Content. https://www.rfc-editor.org/rfc/rfc9110
- RFC 8259, *The JSON Data Interchange Format*: §6 Numbers (interoperability and IEEE 754). https://www.rfc-editor.org/rfc/rfc8259
- RFC 9562, *Universally Unique IDentifiers (UUIDs)*. https://www.rfc-editor.org/rfc/rfc9562
- RFC 9413, *Maintaining Robust Protocols* (on Postel's law and tolerance). https://www.rfc-editor.org/rfc/rfc9413

Python

- `decimal`: contexts, rounding modes and `quantize`. https://docs.python.org/3/library/decimal.html
- Floating-Point Arithmetic: Issues and Limitations. https://docs.python.org/3/tutorial/floatingpoint.html
- `uuid`: UUID objects according to RFC 9562. https://docs.python.org/3/library/uuid.html

Pydantic

- Strict mode. https://docs.pydantic.dev/latest/concepts/strict_mode/
- Conversion table (what lax and strict accept). https://docs.pydantic.dev/latest/concepts/conversion_table/
- Fields and numeric / decimal constraints. https://docs.pydantic.dev/latest/concepts/fields/
- Model configuration, `extra`. https://docs.pydantic.dev/latest/api/config/
- Validation error types (`missing`, `int_type`, `extra_forbidden`, ...). https://docs.pydantic.dev/latest/errors/validation_errors/

FastAPI

- Request body. https://fastapi.tiangolo.com/tutorial/body/
- Path parameters. https://fastapi.tiangolo.com/tutorial/path-params/
- Query parameter models (forbidding extra query parameters). https://fastapi.tiangolo.com/tutorial/query-param-models/

- Response model, return type. https://fastapi.tiangolo.com/tutorial/response-model/
- Handling errors. https://fastapi.tiangolo.com/tutorial/handling-errors/

Security

- OWASP API Security Top 10 (2023), API3: Broken Object Property Level Authorization. https://owasp.org/API-Security/editions/2023/en/0xa3-broken-object-property-level-authorization/
- OWASP Mass Assignment Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html